Exact Equilibrium Solutions for the Board Game *Football Strategy*: A Full-Game Backward-Induction Solver

An Extension of McCulloch's Game-Theoretic Agent

George Kyrollos

Abstract

McCulloch [1] demonstrated that each individual scrimmage play in Avalon Hill's board game *Football Strategy* constitutes a two-player zero-sum matrix game, and designed an agent that solves each play in isolation using mixed-strategy Nash equilibrium. To make the agent sensitive to game context—field position, down, distance, score, and clock—McCulloch augmented the raw yardage payoffs with hand-crafted heuristic bonuses. We replace that heuristic layer entirely. By modeling the complete game as a finite-horizon two-player zero-sum stochastic game and applying exact backward induction, every matrix-game entry becomes the true equilibrium continuation value of the resulting game state. The solver determines optimal play across all ∼30 million reachable states of the fourth quarter, selecting between punt, field goal, and scrimmage plays based on actual win probability rather than proxy rewards. Each team's three timeouts per half are modeled as a pure-strategy sequential sub-game resolved after each play, so the solution reflects optimal timeout usage. We describe the mathematical formulation, state-space representation, solution algorithm, and computational architecture, and compare the approach explicitly to McCulloch's method.

1 Introduction

Football Strategy (Avalon Hill, 1969) is a two-player board game that abstracts American football into a sequence of simultaneous-move interactions. On each scrimmage play the offense secretly selects one of up to 22 actions while the defense simultaneously selects one of ten cards (A–J); the intersection on a printed chart determines the result—yardage, an incomplete pass, a fumble, an interception, a penalty, or a score. Because both players choose without observing the opponent's action, the play is exactly a $|O| \times 10$ two-player zero-sum matrix game.

McCulloch [1] was the first to exploit this structure algorithmically. His agent solves each matrix game using linear programming to obtain a mixed-strategy Nash equilibrium. The key challenge is assigning *utility* to each chart outcome: a raw yardage gain of +7 is not inherently better or worse than a gain of +4 without knowing the down, distance, score, clock, and field position. McCulloch addressed this with a set of hand-tuned bonuses—for first downs, touchdowns, turnovers, and fourth-down conversions—that effectively map game states to a scalar reward. While principled in spirit, these heuristics are necessarily approximate and do not guarantee that the agent's play is optimal for the actual game of *Football Strategy*.

Our contribution is to eliminate the heuristic layer completely. Instead of assigning an ad-hoc scalar to each outcome, we assign it the *exact equilibrium continuation value* of the full game state it produces. This requires solving ∼30 million linked matrix games simultaneously via backward

induction over the complete finite game tree. The result is a provably optimal strategy—in the sense of the minimax theorem for finite two-player zero-sum stochastic games—for the fourth quarter of *Football Strategy*.

2 McCulloch's Approach and Its Limitations

McCulloch's agent operates as follows. At each scrimmage state it constructs a payoff matrix $A \in \mathbb{R}^{|O| \times 10}$ where entry $A_{a,b}$ is the utility assigned to the outcome produced by offensive action a and defensive card b. The matrix game $\max_{\sigma_O} \min_{\sigma_D} \sigma_O^\top A \, \sigma_D$ is then solved via LP to obtain mixed strategies.

The utility function $A_{a,b}$ is where the approximation lives. McCulloch converts each chart result into a scalar by applying additive bonuses:

- A first down earns a bonus proportional to the fraction of the field gained.
- A touchdown earns a large positive bonus scaled by the score context.
- An interception or fumble lost earns a large negative bonus.
- A fourth-down conversion earns a bonus; failing on fourth down earns a penalty.
- Clock and score interact only through the bonus magnitudes, which are set by hand.

The limitations of this approach are fundamental rather than incidental:

Heuristics cannot capture non-monotone value. The value of gaining 10 yards on first-and-10 at midfield in the final minute while trailing by 3 is qualitatively different from the value of the same play with ten minutes remaining. No fixed additive bonus captures this interaction without effectively re-solving the game.

Optimal fourth-down decisions require global knowledge. Whether to punt, attempt a field goal, or go for it on fourth down depends on score, clock, and field position in ways that require comparing continuation values under each option. McCulloch's agent applies fixed cutoffs; the correct decision is state-dependent and only determinable via backward induction.

End-of-half clock management is inaccessible. The strategic value of running out the clock, spiking the ball, or calling a timeout is zero under heuristics but central to real play. Capturing it requires the value function to be defined over clock states.

3 Formal Game Model

3.1 Terminal Utility

We model *Football Strategy* as a finite-horizon two-player zero-sum stochastic game. Let $\delta \in \mathbb{Z}$ denote the score differential (Team 1 minus Team 2) at the end of the game. The terminal utility is

$$U(\delta) = \operatorname{sgn}(\delta) \in \{-1, 0, +1\},$$

so the objective is to maximize win probability, not expected point margin. This is the correct objective for a game whose outcome is win, loss, or draw.

3.2 State Representation

A complete game state must encode every quantity that affects future optimal play. We use a *symmetric possession* convention: the state is always written from the perspective of the team currently in possession of the ball, so field position $x \in [1, 99]$ is always measured in yards from the current offense's own goal line ($x = 0$: own goal; $x = 100$: opponent's goal).

Definition 1 (Scrimmage state)**.** *A scrimmage state is a tuple*

$$s = (q,\, \tau,\, x,\, d,\, y,\, \delta,\, h,\, \theta_1,\, \theta_2),$$

where:

- $q \in \{1, 2, 3, 4\}$ *is the quarter;*
- $\tau \in \{0, 1, \ldots, 60\}$ *is the number of 15-second ticks remaining in the quarter;*
- $x \in \{1, \ldots, 99\}$ *is ball position (yards from current offense's own goal);*
- $d \in \{1, 2, 3, 4\}$ *is the current down;*
- $y \in \{1, \ldots, 30\}$ *is yards to first down (capped at 30);*
- $\delta \in \{-35, \ldots, 35\}$ *is the score differential from the current offense's perspective (clipped at* ± 35*);*
- $h \in \{-1, 0, +1\}$ *encodes which team receives the second-half kickoff (*$h = 0$ *in Q3/Q4 when this is already resolved); and*
- $\theta_1, \theta_2 \in \{0, 1, 2, 3\}$ *are the remaining timeouts for the current offense and defense, respectively.*

The full game starts at $\theta_1 = \theta_2 = 3$.
In addition to scrimmage states, the game includes *kickoff states* (q, τ, δ, h) and *safety-kick states* (q, τ, δ, h).

Possession symmetry. Because the state is always expressed from the current offense's viewpoint, a possession change transforms the state as

$$x \;\mapsto\; 100 - x, \qquad \delta \;\mapsto\; -\delta, \qquad h \;\mapsto\; -h,$$

and the continuation value is negated (since the current maximizer becomes the minimizer). This halves the effective state space compared to a possession-explicit encoding.

3.3 Transition Kernel

The transition kernel $P(s' \mid s, a, b)$ is determined by:

1. **Chart lookup:** the cell at row b (defense card), column a (offense play) of the selected offense chart.

2. **Outcome parsing:** chart cells encode one of: a deterministic yardage gain or loss (possibly out-of-bounds), an incomplete pass, a fumble, an interception with return yards, a long gain (LG), a penalty, a loss-of-down, or an OR-choice between two outcomes resolved by the favored team.

3. **Game-rule application:** yardage is added to x; if $x \geq 100$ a touchdown results; if $x \leq 0$ a safety results; first downs reset d and y; fourth-down failures transfer possession.

4. **Clock advancement:** τ is decremented by the tick cost of the outcome type (Table 1), subject to the two-minute automatic stoppage at $\tau = 8$ in Q2 and Q4.

Table 1: Time consumed per outcome category (15 sec per tick).

Outcome type	Time (sec)	Ticks
In-bounds gain of 0–19 yards	30	2
In-bounds gain of ≥ 20 yards	45	3
Any loss	30	2
Out-of-bounds play	15	1
Incomplete pass	15	1
Interception	30	2
Fumble	15	1
Penalty	15	1
Kickoff / field goal / punt	15	1
Extra point (PAT)	0	0

Long Gain. Chart cells marked `LG` trigger a probabilistic outcome drawn from the distribution in Table 2, which was derived from the printed Long Gain table. LG outcomes always consume 3 ticks.

Table 2: Long Gain distribution.

Gain (yards)	Probability
+30, +35, +40, +45, +50	$\frac{1}{6}$ each
+60, +70, …, +110	$\frac{1}{36}$ each

Field goals. A field goal attempt is legal on any down when $x \geq 55$ (within 45 yards of the opponent's goal line). Success probabilities are distance-based (Table 3).

Extra points. After every touchdown the scoring team attempts a PAT with $P(\text{good}) = \frac{34}{36}$. PATs consume zero clock time.

Table 3: Field goal success probabilities by distance.

Distance to goal line (yards)	P(good)
1–12	$\frac{32}{36}$
13–22	$\frac{28}{36}$
23–32	$\frac{18}{36}$
33–38	$\frac{12}{36}$
39–45	$\frac{2}{36}$

4 Solution Algorithm

4.1 The Core Idea

The fundamental observation is that *Football Strategy* is finite: the clock always ticks forward, so the game must end. This means we can work backwards from the final whistle. At the last possible moment (clock at zero), the outcome is just win, loss, or draw — the terminal utility $U(\delta) = \text{sgn}(\delta)$. One tick earlier, each team chooses optimally given that they know exactly what will happen at clock zero. One tick before that, they choose knowing what will happen one tick from now. Repeat all the way back to kickoff. This is backward induction, and it gives the exact equilibrium of the game.

4.2 Solving Each Game State

At each scrimmage state s, both teams choose simultaneously and without seeing each other's decision — the offense picks a play number, the defense picks a card, and the result is read from the chart. This is exactly a finite two-player zero-sum matrix game. The payoff matrix $A^{(s)}$ has one row per legal offensive action and ten columns (one per defense card A–J), with entry

$$A^{(s)}_{a,b} = \sum_{s'} P(s' \mid s, a, b)\, V(s'),$$

the expected game value of the position that results from the play. Since we solve from low clock to high clock, $V(s')$ is always already known.

The equilibrium value of state s is the solution to the minimax problem:

$$V(s) = \max_{\sigma_O} \min_{\sigma_D} \sigma_O^\top A^{(s)} \sigma_D. \tag{1}$$

By the minimax theorem, there is always a well-defined value and optimal (possibly mixed) strategies for both sides.

4.3 Finding the Equilibrium: Saddle Points and Linear Programming

For many game states there is a single dominant offensive play — one that is at least as good as every other play regardless of what the defense calls. We call this a *saddle point*: the maximum of the row-minimum values equals the minimum of the column-maximum values,

$$\max_a \min_b A_{a,b} = \min_b \max_a A_{a,b}.$$

When this holds, the optimal strategy is pure (no randomisation needed), and we read the answer directly off the matrix in $O(|O| \cdot |D|)$ time without any LP call. This is not an approximation — it is the exact Nash equilibrium, guaranteed by the saddle-point theorem. In practice this shortcut fires on roughly 40–60% of game states, substantially reducing total solve time.

When no saddle point exists, the equilibrium is a mixed strategy found by solving the row player's (offense's) LP:

$$\begin{aligned} \text{maximize} \quad & v \\ \text{subject to} \quad & \sum_a p_a \, A^{(s)}_{a,b} \geq v \quad \forall b, \\ & \textstyle\sum_a p_a = 1, \quad p_a \geq 0. \end{aligned}$$

The optimal $v^* = V(s)$ and p^* is the offense's equilibrium mixed strategy. Both teams' strategies are computed using the HiGHS solver via `scipy.optimize.linprog`.

4.4 Backward Induction and Forward Reachability

Because the clock only moves forward, all states at time τ depend only on states at $\tau' < \tau$ — there are no circular dependencies. This means we can solve the whole game in a single sweep: process the states at $\tau = 1$ first (only one tick of game remaining), then $\tau = 2$, all the way up to $\tau = 60$. Within each quarter, every state at a given τ is completely independent of every other state at the same τ, so they can be solved in parallel. After each quarter is done, we seed the next quarter from the quarter boundary values and repeat.

Not every state in the theoretical state space is reachable from the opening kickoff. Before solving, we run a forward BFS from the start state to enumerate only the states that can actually occur under any sequence of plays. This reduces the number of states to solve from hundreds of millions to approximately 30 million for Q4.

Algorithm 1 Full-game backward induction solver

Require: Offense chart C, punt chart P, start state s_0
$\mathcal{S} \leftarrow \textsc{ForwardBFS}(C, P, s_0)$ ▷ Only reachable states
for $q \in \{4, 3, 2, 1\}$ **do**
  Apply quarter-boundary seeding (e.g. halftime kickoff)
  for $\tau \in \{1, 2, \ldots, 60\}$ **do**
    for each reachable state s at (q, τ) **in parallel do**
      if s is a scrimmage state **then**
        Build $A^{(s)}$; check for saddle point
        $V(s) \leftarrow$ saddle-point value or LP value of $A^{(s)}$
      else if s is a kickoff or safety-kick state **then**
        $V(s) \leftarrow$ closed-form expected value
      end if
    end for
  end for
end forreturn V

4.5 Value Storage

Rather than a hash map over 30 million keys, values are stored in dense multi-dimensional NumPy arrays indexed directly by state coordinates, giving O(1) lookup:

- Q3/Q4 scrimmage: shape (2, 61, 99, 4, 30, 71), approximately 823 MB. No h dimension because $h = 0$ in the second half.
- Q1/Q2 scrimmage: shape (2, 61, 99, 4, 30, 71, 2), approximately 1.65 GB.
- Kickoff and safety-kick arrays: negligible size.

Total memory: approximately 2.47 GB. Unsolved slots are initialised to `NaN`; any read of an unsolved slot raises an error immediately, catching dependency bugs.

5 Computational Implementation

Solving ∼30 million matrix games is not expensive in principle — each game is tiny ($\leq 22 \times 10$) — but doing it 16 times over (once per timeout-count combination) requires care. This section describes the engineering decisions that make the solve tractable.

5.1 Score Differential Clipping

The score differential δ (offense minus defense) is theoretically unbounded, but a lead of more than 35 points in a single quarter is effectively insurmountable. We clip δ to the range $[-35, 35]$, giving 71 distinct values. This is not just an approximation made for convenience: the terminal utility $U(\delta) = \text{sgn}(\delta)$ is flat outside $\{-1, 0, +1\}$, so states with $|\delta| > 35$ already have a certain outcome and their values propagate trivially. Clipping makes the state space finite and dense-array-representable without affecting the equilibrium strategies in any reachable game situation.

5.2 Symmetric Possession Representation

Rather than tracking both field position from Team 1's perspective *and* which team has the ball, we always write the state from the current offense's point of view. Field position x is always yards from the *current offense's* own goal. When possession changes, we flip $x \to 100 - x$, $\delta \to -\delta$, $h \to -h$, and negate the continuation value. This halves the state space and means a single value table covers both teams — there is no asymmetry to track.

5.3 Pure-Strategy Saddle-Point Shortcut

Before calling the LP solver on a payoff matrix A, we check whether a saddle point exists. A saddle point is a position in the matrix that is simultaneously the smallest value in its row and the largest value in its column — in other words, a play the offense always wants to run regardless of what the defense calls, and a card the defense always wants to call regardless of what the offense runs.

Concretely: if the maximum of the row-minimum values equals the minimum of the column-maximum values,

$$\underbrace{\max_a \min_b A_{a,b}}_{\text{maximin}} = \underbrace{\min_b \max_a A_{a,b}}_{\text{minimax}},$$

then the saddle point is a pure Nash equilibrium and no LP is needed. The game value is exactly the saddle-point entry. This check costs $O(22 \times 10)$ arithmetic operations, compared to the LP which requires hundreds of simplex iterations. In practice it fires on roughly 40–60% of game states (most clearly in short-yardage situations and end-of-game scenarios), roughly halving total solve time.

5.4 Fork-Based Parallelism

Two levels of parallelism are exploited.

Within a τ layer. States at the same (q, τ) are completely independent — they read values only from already-solved lower-τ states. The implementation distributes them across CPU cores using Python's `multiprocessing.Pool` with the `fork` start method. Forking is critical: it makes the full ∼2.5 GB value table available to every worker at zero serialization cost (the operating system shares pages copy-on-write), so workers pay only for the small number of pages they actually write.

Across timeout-count combinations. Combinations (θ_1, θ_2) with the same total $\theta_1 + \theta_2$ are independent of each other — they read only from already-solved combinations with smaller totals. Rather than solving them sequentially, we fork one process per combination at each level and solve them simultaneously. With 16 workers and 4 combinations at the most populated level, this gives up to 4× speedup on the timeout solve. Workers are split evenly: if k combinations run in parallel each gets $\lfloor n_{\text{workers}}/k \rfloor$ state-level workers.

5.5 Runtime and Memory

The Q4 base solve (no timeouts remaining) runs in approximately 8 hours on a 16-core machine. With the saddle-point shortcut and parallel combo execution the full 16-table solve is estimated to take 1–2 days on the same hardware. Peak RAM usage is approximately $3 \times 823\,\text{MB} \approx 2.5\,\text{GB}$ for the active value tables at any one time; completed tables are saved to disk and evicted.

6 Timeout Decisions

Each team is entitled to three timeouts per half. Calling a timeout after a play reduces the clock charge according to Table 4.

Table 4: Clock time counted after a timeout is called, by original play duration.

Original play duration	Ticks without TO	Ticks after TO
45 sec (long gain)	3	1
30 sec (normal play)	2	0
15 sec (OOB / penalty)	1	0

A timeout can save clock ticks after any play type. The most critical case is a 1-tick play (out-of-bounds, incomplete, or penalty) when the clock is about to expire: calling a timeout reduces the charge to 0 ticks, preserving the remaining time entirely and allowing one more snap.

After a 2-tick play the savings are even larger (2 ticks saved), and after a 3-tick long gain the charge drops from 3 to 1 tick.

6.1 The Post-Play Timeout Sub-Game

After each play completes with tick cost k, either team may call a timeout to reduce the clock charge to $k' < k$. Let

$$\begin{aligned} a &= V^{(\theta_1-1,\,\theta_2)}(s',\,k') \quad \text{(offense calls)},\\ b &= V^{(\theta_1,\,\theta_2-1)}(s',\,k') \quad \text{(defense calls)},\\ c &= V^{(\theta_1,\,\theta_2)}(s',\,k) \quad \text{(neither calls)}. \end{aligned}$$

Because this is a zero-sum game, reduced clock time can benefit at most one team, so both calling simultaneously is never rational. Since at most one team will call, the order of decisions is irrelevant: whichever team benefits simply calls, and the other does not. The sub-game value is therefore

$$V_{\text{TO}} = \max\big(a,\ \min(b,\,c)\big), \tag{2}$$

which replaces the plain $V^{(\theta_1,\theta_2)}(s',k)$ lookup in the outer LP.

6.2 Layered Solve Architecture

Adding timeouts to the state tuple would multiply the state space by $4^2 = 16$. Instead, we maintain 16 separate value tables $V^{(\theta_1,\theta_2)}$, each with the same array layout as the no-timeout base (≈ 823 MB per table, ≈ 13 GB total). The tables are solved in order of increasing $\theta_1 + \theta_2$:

$$(0,0) \ \to\ (1,0),\,(0,1) \ \to\ (2,0),\,(1,1),\,(0,2) \ \to\ \cdots \ \to\ (3,3).$$

The $(0,0)$ table (no timeouts remaining) is solved first and serves as the base case. Each subsequent (θ_1,θ_2) requires a full backward-induction pass, with the post-play sub-game looking up already-solved predecessor tables. The game-relevant output is $V^{(3,3)}$: both teams begin with three timeouts.

7 Comparison to McCulloch's Method

Table 5 summarizes the principal differences between McCulloch's approach and ours.

The most consequential difference is the matrix entry. In McCulloch's formulation, $A^{(s)}_{a,b}$ is an approximate scalar that proxies game value. In ours, it is $Q_s(a,b) = \mathbb{E}[V(s') \mid s,a,b]$, which is the exact expected value of the resulting position under equilibrium play from that position onward. Consequently, our equilibrium mixed strategy is a Nash equilibrium for the *game of Football Strategy* itself, not merely for the local matrix game with surrogate payoffs.

On the correctness of heuristic strategies. It is in principle possible that McCulloch's heuristics happen to produce strategies close to the true equilibrium; whether this is so is an empirical question that our solver can now answer directly, by comparing the heuristic mixed strategies to the exact equilibrium strategies at each state.

Table 5: McCulloch's heuristic agent vs. the exact backward-induction solver.

Aspect	McCulloch (2004)	This work
Matrix entry value	Heuristic bonus on raw yardage	Exact continuation value $Q_s(a, b)$
Optimality guarantee	Local Nash per play; global not guaranteed	Global Nash for the full game
Clock	Not modeled explicitly	Exact 15-sec tick accounting, two-minute warning, timeouts
Fourth-down decisions	Fixed heuristic cutoffs	Optimal from $V(s)$ for punt/FG/go
Score sensitivity	Bonus scaling by score margin	Terminal utility $U(\delta) \in \{-1, 0, +1\}$
Field goal	Heuristic range threshold	Exact probability by distance
Kickoff / safety kick	Not modeled as strategic decisions	Full stochastic transitions; onside kick option included
Punts	Fixed utility for receiving position	Optimal punt vs. go-for-it trade-off
Timeouts	Not modeled	Pure-strategy sequential sub-game; 16-table layered solve
State space solved	Each play independently	$\sim$30M states $\times$ 16 TO combos
Tunable parameters	Several heuristic constants	None

8 Red-Zone Play Restrictions and Chart Parsing

The Ball Control Offense Plays Chart restricts legal play sets near the opponent's goal line. In offense-relative coordinates:

- Plays 17–20 are illegal when $x \geq 80$ (within the opponent's 20).
- Plays 13–16 are illegal when $x \geq 90$ (within the opponent's 10).

The chart is parsed cell-by-cell into a typed intermediate representation before solving. Each cell token is classified as one of: YARDS, INCOMPLETE, FUMBLE, LONGGAIN, INTERCEPTION(r), PENALTY(y), LOSSOFDOWN(y), or an OR-choice CHOICE($\text{branch}_1, \text{branch}_2, \text{resolver}$) where `resolver` $\in \{\text{offense}, \text{defense}\}$ indicates which team selects the better branch. This separation between parsing and solving simplifies both the transition engine and the LP construction.

A subtle rule applies to interceptions with large negative return yards (the intercepting defender runs toward their own end zone): if the resulting field position exceeds the physical boundary of the defender's end zone (more than 110 yards from the offense's own goal line in offense-relative coordinates), the play is treated as an incomplete pass rather than a touchback.

9 Scale and Computational Results

The fourth-quarter solve enumerates approximately 30.9 million reachable states across 60 clock layers via forward BFS. Of these, approximately 615,000 are unique *scrimmage core* states (the (x, d, y, δ, h) component), with each appearing at multiple clock values. Each scrimmage state requires solving a matrix LP with at most 22 rows (offense actions) and 10 columns (defense cards).

The no-timeout base ($\theta_1 = \theta_2 = 0$) runs in approximately 8 hours on an 8-core machine; the full 16-table solve requires approximately $16\times$ that time. Each value table is stored in approximately 823 MB as a dense NumPy array.

The value $V(s_0)$ at the initial kickoff state ($q = 4, \tau = 60, \delta = 0$) gives the exact win probability for the team kicking off at the start of the fourth quarter under a tied score. Any deviation from the equilibrium mixed strategies by either player can only decrease that player's win probability.

References

[1] Sean McCulloch. A game-theoretic intelligent agent for the board game *Football Strategy*. *Unpublished manuscript*, 2004.

[2] John von Neumann. Zur Theorie der Gesellschaftsspiele. *Mathematische Annalen*, 100(1):295–320, 1928.

[3] Lloyd Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953.

[4] Martin Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[5] Julian Hall, Ivet Galabova, Leona Gottwald, and Michael Feldmeier. HiGHS: High performance software for linear optimization. https://highs.dev, 2023.